

Übung 10 Maschinensprache

UE 10.0 Theorie

1. Wo wird der Mikrocode gespeichert? Wo steht ein Programm in Maschinensprache?
2. Welche Aspekte von Mikrocode werden in Maschinensprache abstrahiert?
3. Könnte ein Programm in Maschinensprache seinen eigenen Code verändern?

UE 10.1

Wir ordnen den allgemeinen CPU Registern R0 bis R7 ab jetzt folgende Assembler-Register zu:

R0	--	IP	Instruction Pointer Register
R1	--	A	Akkumulator A
R2	--	B	Akkumulator B
R3	--	I	LOOP-Indexregister
R4	--	X	Hilfsregister AUX
R5	--	IO	Input/Output Port Register
R6	--	DP	Data Pointer Register
R7	--	SP	Stack Pointer Register

Schreiben Sie Mikrocoderroutinen für einen Interpretierer, die folgende Maschinensprache-Befehle implementieren:

RESET

Opcode : **00H**

Initialisierungsroutine die alle Register und MAR auf 0 initialisiert.
(Schreibweise: 0 → R0, ..., R7, MAR)

LOAD-INC-EXECUTE

Opcode: **01H**

Load-Increment-Execute-Zyklus

Um gewünschte Maschinensprachebefehle in den Mikrocodesimulator aufnehmen zu können, muss die Spezifikation des Load-Inc-Execute-Zyklus bekannt sein. Load-Inc-Execute Routine repräsentiert eine Interpreterschleife, die den nächsten Opcode aus dem RAM liest, den Programmzähler (IP) erhöht und zur entsprechenden Mikroroutine, die den Opcode implementiert, verzweigt.

Die Interpreterschleife für Maschinensprache kann wie folgt beschrieben werden:

- holt Program-Counter-Inhalt aus dem Register R0 (=IP) und leitet ihn ins MAR, (Schreibweise: IP → MAR),
- Im nächsten Takt wird der RAM aufgefordert, 1 Byte an der Adresse MAR, aus dem Speicher zu holen, RAM-Status lesend, (Schreibweise: [MAR] → MDR),
- der OpCode kann von MDR → COP gleich weitergeleitet werden
- IP (R0) um 1 inkrementieren, (Schreibweise: IP+1 → IP),
- Optional kann IP → MAR gleich wieder gesetzt werden damit der OpCode an dieser Stelle gleich sein Argument lesen kann
- Springe im ROM absolut zu 4xCOP, zur Implementierung des OpCodes

Daraus ergibt sich für die Programmierung eines Maschinensprachebefehls an den Mikrocode-Adressen 4xCOP im ROM:

- Bei einem Mehrbyte-Befehl kann sofort mit dem Lesen der restlichen Bytes begonnen werden, da IP+1 im MAR steht.
- Der IP muss pro Argument um 1 erhöht werden.
- Der Rücksprung in zur LOAD-INC-EXEC Prozedur muss zur Adresse 004h (Segment Nr 1) erfolgen.

MOV A, [m]

Opcode: **02h**

Der Inhalt (1 Byte) der RAM Speicherzelle *m* (1 Byte-Adresse) wird in das Register A (R1) kopiert.

(Schreibweise: [m]->MDR, MDR → R1)

Ein 2- Byte Maschinenbefehl; Erstes Byte ist der Opcode selbst, das zweite Byte ist das Argument *m*. Der Befehl besitzt ein Argument, nämlich die 1 Byte umfassende Adresse [m] von jener Speicherzelle deren Wert ins Register A (R1) gelesen werden soll.

Der komplette Befehl MOV A, [m] wird aus den zwei aufeinanderfolgenden Bytes *Opcode* und *m* bestehen.

MOV B, [m]

Opcode: **03h**

Der Inhalt (1 Byte) der RAM Speicherzelle *m* (1 Byte-Adresse) wird in das Register B (R2) kopiert.

(Schreibweise: [m]->MDR, MDR → R2)

Ebenfalls ein 2- Byte Maschinenbefehl;

ADD A, B

Opcode: **04H**

Der Inhalt von Register B wird zu dem Inhalt von Register A addiert.

(Schreibweise: R1+R2 → R1)

MOV [m], A

Opcode: **05H**

Der Inhalt (1 Byte) von Register A wird in die RAM Speicherzelle *m* (1 Byte-Adresse) kopiert.

(Schreibweise: R1 → MDR, MDR → [m])

Wieder ein 2-Byte Maschinenbefehl

UE 10.2

Erstellen Sie ein Programm in Maschinensprache das folgende Aufgabe erledigt:

[020H] + [022H] + [024H] → [026H]

1. Inhalt (1 Byte z.B. 3) der RAM Speicherzelle [020H] wird gelesen und mit dem Inhalt (1 Byte z.B. 4) der RAM Speicherzelle [022H] addiert.
2. Das Ergebnis wird mit dem Inhalt (1 Byte z.B. 2) der RAM Speicherzelle [024H] addiert.
3. Das Ergebnis wird im RAM auf Adresse [026H] als 1 Byte abgespeichert.